```
// g++ -O3 -std=c++17 -fopenmp -fno-fast-math -ffp-contract=off \
//     crank_verify_certified.cpp -lgmp -o crank_verify
// ./crank_verify --threads 8 --output full_crank_results.csv
// The default run generates the partition table from scratch.
// Paper (4.2): 72 <= n < 10^6, 1 <= m <= min(n-72, floor(n/3)).
// For negative m, crank symmetry gives the same inequality.
// This program exhaustively checks this finite residual region.

#include <gmp.h>
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cfloat>
#include <cfenv>
#include <cstdio>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
#include <omp.h>

struct Z
{
    mpz_t z;

    Z()
    {
        mpz_init(z);
    }

    ~Z()
    {
        mpz_clear(z);
    }

    Z(const Z &) = delete;
};

using Clock = std::chrono::steady_clock;

double seconds(Clock::time_point a)
{
    return std::chrono::duration<double>(Clock::now() - a).count();
}

std::vector<Z> p(1000000);

void coefficient(mpz_t out, int m, int n, mpz_t tmp)
{
    mpz_set_ui(out, 0);
    for (int r = 1;; r++)
    {
        int x = n - r * m - r * (r - 1) / 2;
        if (x < 0)
            break;
        if (x >= r)
            mpz_sub(tmp, p[x].z, p[x - r].z);
        else
            mpz_set(tmp, p[x].z);
        if (r & 1)
            mpz_add(out, out, tmp);
        else
            mpz_sub(out, out, tmp);
    }
}
```

```
struct Ball
{
    long double value, error;
};

std::vector<long double> pf(1000000);

// With n < 10^6, at most 1414 crank summands are nonzero.
// mpz_get_d_2exp truncates positive integers toward zero at 53 bits.
// For u <= 2^-64, (2^-52 + gamma_4000)/(1-gamma_4000) < 2^-50.
// Thus 2 * sa * 2^-50 bounds the prefix error, including rounding.
// The omitted tail is at most 1414 * p[xn] < 4000 * pf[xn].
Ball coefficient_ball(int m, int n)
{
    long double value = 0, sa = 0;
    for (int r = 1;; r++)
    {
        int x = n - r * m - r * (r - 1) / 2;
        if (x < 0)
            break;
        long double a = pf[x], b = x >= r ? pf[x - r] : 0;
        long double term = a - b;
        sa += a + b;
        if (r & 1)
            value += term;
        else
            value -= term;
        int rn = r + 1;
        int xn = n - rn * m - rn * (rn - 1) / 2;
        if (xn >= 0)
        {
            long double tail = 4000 * pf[xn];
            if (tail < (std::abs(value) * 0x1p-48L))
                return {value, 2 * ((sa * 0x1p-50L) + tail)};
        }
    }
    return {value, 2 * (sa * 0x1p-50L)};
}

// Certify M(m,n)^2 - M(m-1,n)*M(m+1,n) >= 0.
// An inconclusive interval always triggers exact GMP integer arithmetic.
long long row_ball(int n, long long &fallback)
{
    int bound = std::min(n - 72, n / 3);
    if (bound < 1)
        return 0;
    Ball le = coefficient_ball(0, n), mi = coefficient_ball(1, n);
    for (int m = 1; m <= bound; m++)
    {
        Ball ri = coefficient_ball(m + 1, n);
        long double square = mi.value * mi.value, prod = le.value * ri.value,
                    D = square - prod;
        long double err = std::max({le.error, mi.error, ri.error});
        long double budget =
            4 * err *
                (std::abs(le.value) + std::abs(mi.value) + std::abs(ri.value) + err) +
            (std::abs(square) + std::abs(prod)) * 0x1p-59L;
        if (!(D > budget))
        {
            fallback++;
            mpz_t a, b, c, tmp, lhs, rhs;
            mpz_inits(a, b, c, tmp, lhs, rhs, NULL);
            coefficient(a, m - 1, n, tmp);
            coefficient(b, m, n, tmp);
            coefficient(c, m + 1, n, tmp);
            mpz_mul(lhs, b, b);
```

```
            mpz_mul(rhs, a, c);
            if (mpz_cmp(lhs, rhs) < 0)
            {
                std::cerr << "FAIL n=" << n << " m=" << m << std::endl;
                std::exit(2);
            }
            mpz_clears(a, b, c, tmp, lhs, rhs, NULL);
        }
        le = mi;
        mi = ri;
    }
    return bound;
}

long long expected_through(int H)
{
    if (H < 72)
        return 0;
    if (H <= 108)
        return (long long)(H - 72) * (H - 71) / 2;
    long long q = H / 3, rem = H % 3;
    return 3 * q * (q - 1) / 2 + (rem + 1) * q - 1260;
}

int main(int argc, char **argv)
{
    int first = 72, last = 999999, threads = 8;
    std::string cache_path, output = "full_crank_results.csv";
    for (int j = 1; j < argc; j++)
    {
        std::string a = argv[j];
        if (j + 1 >= argc)
        {
            std::cerr << "Missing argument";
            return 7;
        }
        if (a == "--first")
            first = std::stoi(argv[++j]);
        else if (a == "--last")
            last = std::stoi(argv[++j]);
        else if (a == "--threads")
            threads = std::stoi(argv[++j]);
        else if (a == "--cache")
            cache_path = argv[++j];
        else if (a == "--output")
            output = argv[++j];
        else
            return 7;
    }
    if (first < 72 || last < first || last >= 1000000 || threads < 1)
        return 7;
    if (FLT_RADIX != 2 || DBL_MANT_DIG != 53 || LDBL_MANT_DIG < 64 ||
        LDBL_MAX_EXP < 8192 || std::fesetround(FE_TONEAREST))
    {
        std::cerr << "Unsupported floating-point environment";
        return 7;
    }
    auto start = Clock::now();
    if (!cache_path.empty())
    {
        FILE *f = fopen(cache_path.c_str(), "rb");
        if (!f)
            return 3;
        for (int n = 0; n < 1000000; n++)
            if (!mpz_inp_raw(p[n].z, f))
            {
                fclose(f);
```

```
            return 3;
        }
    fclose(f);
}
else
{
    mpz_set_ui(p[0].z, 1);
    for (int n = 1; n < 1000000; n++)
    {
        for (int r = 1;; r++)
        {
            // Euler pentagonal exponents, unchanged for crank counting.
            int g = r * (3 * r - 1) / 2;
            if (g > n)
                break;
            if (r & 1)
            {
                mpz_add(p[n].z, p[n].z, p[n - g].z);
                if (g + r <= n)
                    mpz_add(p[n].z, p[n].z, p[n - g - r].z);
            }
            else
            {
                mpz_sub(p[n].z, p[n].z, p[n - g].z);
                if (g + r <= n)
                    mpz_sub(p[n].z, p[n].z, p[n - g - r].z);
            }
        }
        if (n % 100000 == 0)
            std::cout << "PARTITION_PROGRESS n=" << n
                      << " elapsed=" << seconds(start) << std::endl;
    }
}
// Independently check the partition recurrence through q^100.
std::vector<Z> small(101);
mpz_set_ui(small[0].z, 1);
for (int part = 1; part <= 100; part++)
    for (int n = part; n <= 100; n++)
        mpz_add(small[n].z, small[n].z, small[n - part].z);
for (int n = 0; n <= 100; n++)
    if (mpz_cmp(p[n].z, small[n].z))
    {
        std::cerr << "Partition cross-check failed";
        return 8;
    }
for (int n = 0; n < 1000000; n++)
{
    if (mpz_sgn(p[n].z) <= 0 || mpz_sizeinbase(p[n].z, 2) > 4000)
        return 8;
    long e;
    double d = mpz_get_d_2exp(&e, p[n].z);
    pf[n] = std::ldexp((long double)d, e);
}
std::cout << "PARTITIONS_READY elapsed=" << seconds(start)
          << " LDBL_MANT_DIG=" << LDBL_MANT_DIG << " LDBL_MAX_EXP=" << LDBL_MAX_EXP
          << " GMP=" << gmp_version << std::endl;
std::ofstream out(output);
if (!out)
{
    std::cerr << "Cannot open output CSV" << std::endl;
    return 10;
}
out << "first,last,comparisons,expected,fallbacks,seconds,cumulative,status\n";
long long cumulative = 0, totalfb = 0;
for (int lo = first; lo <= last; lo += 10000)
{
    int hi = std::min(last, lo + 9999);
```

```
        long long checked = 0, fb = 0;
        auto st = Clock::now();
#pragma omp parallel for num_threads(threads) reduction(+ : checked, fb)              \
    schedule(dynamic, 1)
        for (int n = lo; n <= hi; n++)
            checked += row_ball(n, fb);
        long long expected = expected_through(hi) - expected_through(lo - 1);
        if (checked != expected)
        {
            std::cerr << "Coverage count mismatch";
            return 9;
        }
        cumulative += checked;
        totalfb += fb;
        double elapsed = seconds(st);
        out << lo << ',' << hi << ',' << checked << ',' << expected << ',' << fb << ','
            << std::setprecision(12) << elapsed << ',' << cumulative << ",PASS\n";
        out.flush();
        if (!out)
        {
            std::cerr << "Cannot write output CSV: " << output << '\n';
            return 10;
        }
        std::cout << "BLOCK_PASS n=" << lo << ".." << hi << " comparisons=" << checked
                  << " fallbacks=" << fb << " cumulative=" << cumulative
                  << " block_seconds=" << elapsed << " elapsed=" << seconds(start)
                  << std::endl;
    }
    long long expected = expected_through(last) - expected_through(first - 1);
    if (cumulative != expected)
        return 9;
    out.close();
    if (!out)
    {
        std::cerr << "Cannot close output CSV: " << output << '\n';
        return 10;
    }
    std::cout << "COMPLETE_PASS first=" << first << " last=" << last
              << " comparisons=" << cumulative << " expected=" << expected
              << " fallbacks=" << totalfb << " seconds=" << seconds(start) << std::endl;
    return 0;
}
```